Hoofdstuk 6

ETL

In hoofdstuk 1 is het datawarehouse de *'single version of the truth'* genoemd. In hoofdstuk 3 is als een van de argumenten om een datawarehouse te bouwen de slechte kwaliteit aangegeven van de gegevens in operationele databases. Blijkbaar denken we in staat te zijn om die slechte kwaliteit te verbeteren tot 'de waarheid' door die gegevens uit de operationele systemen te halen en op te slaan in het datawarehouse. Absoluut geen kleine prestatie. Iedereen kent het cliché *'garbage in, garbage out'*, maar hier willen we het omgekeerde doen: de waarheid maken van de troep die binnenkomt.

Bovenstaande uitdaging is het ETL-proces in een notendop.

6.1 Inleiding

ETL staat voor Extract, Transform en Load. De Extract staat voor het lezen van gegevens uit operationele systemen en andere mogelijke bronnen. De Load staat voor het vullen van het datawarehouse met gegevens. De Transform in het midden moet de ingelezen gegevens zo manipuleren dat de organisatie uiteindelijk bereid is de informatie die uit het datawarehouse komt, te accepteren als de waarheid. Dat is al een nuance ten opzichte van dat het de waarheid is, maar nog altijd geen kleine opgave. Als we het hier over het datawarehouse hebben, bedoelen we over het algemeen de datamarts waar eindgebruikers mee werken. In het echte datawarehouse zitten de gegevens vaak in hun originele vorm.

Op Wikipedia staat de volgende definitie van ETL: 'ETL is een groep technologieën die veelal gebruikt wordt bij de koppeling tussen systemen, waarbij er gestreefd wordt naar een minimale technische en semantische koppeling. Het is een batchproces dat regelmatig gebruikt wordt.'

In het kader van dit boek kan de koppeling tussen systemen op twee manieren worden opgevat. Enerzijds kennen de meeste Business Intelligence-oplossingen meer bronnen. Er worden dus meer bronnen gekoppeld. Anderzijds is het de koppeling tussen operationele systemen als CRM-systemen en het datawarehouse.

ETL is een groep technologieën die veelal gebruikt wordt bij de koppeling tussen systemen, waarbij er gestreefd wordt naar een minimale technische en semantische koppeling. Het is een batchproces dat regelmatig gebruikt wordt.

De minimale koppeling waar de definitie over spreekt is belangrijk voor de onderhoudbaarheid van het ETL-proces. De verschillende bronsystemen kunnen allemaal technisch verschillend zijn: relationele databases uit andere DMBS'en, tekstbestanden, RSS feeds en zo verder. Dit mag geen probleem opleveren op technisch vlak. De semantische koppeling zit meer richting de BI life cycle waar in hoofdstuk 2 over is gesproken. De organisatie mag anders tegen de wereld gaan aankijken, een kolom blijft een

kolom. Verder moeten we er rekening mee houden dat bronsystemen kunnen veranderen bij de implementatie van nieuwere versies van de database. We moeten er naar streven dat deze veranderingen minimale impact hebben op de ETL en daarmee op het datawarehouse.

In dit hoofdstuk zullen we de verschillende aspecten van ETL op een rij zetten en enkele voorbeelden uitwerken. We beginnen echter met te kijken naar Master Data, een eerste stap in het omgaan met verschillende operationele databases.

6.2 Master Data Management

In de inleiding en in hoofdstuk 1 is gesproken over het bestaan van meer dan één operationeel systeem met ieder zijn eigen database(s) als een probleem dat (mede) leidt tot Business Intelligence-initiatieven. Om te zorgen voor eenduidige rapportages brengen we alle gegevens onder in één centraal datawarehouse. Zolang alle rapporten dit datawarehouse gebruiken als bron, is er eenduidigheid over alle rapportages. Er blijven echter twee problemen over:

- De operationele processen hebben nog steeds last van de niet-eenduidige gegevens.
- Tijdens de ETL moet gekozen worden 'welke waarheid' in het datawarehouse wordt opgenomen.

Op operationeel niveau kunnen er veel dingen fout gaan als verschillende processen en/of afdelingen werken met hun eigen gegevens.

> **Voorbeeld**
> Een huiseigenaar wordt door zijn bank gebeld met een aanbieding voor een nieuwe hypotheek, die gegarandeerd voordeliger is dan zijn huidige. Een paar maanden daarvoor heeft deze persoon echter zijn laatste hypotheekbetaling gedaan en zijn huis is volledig afgelost en vrij op naam. Blijkbaar is de database van de marketingafdeling een andere dan die van de hypotheekafdeling!

Voor het ETL-proces zijn er eigenlijk twee problemen:

- Twee verschillende systemen geven tegenstrijdige informatie.
- Voor twee verschillende systemen ziet de waarheid er anders uit.

Het eerste punt is hetzelfde als het voorbeeld. Uit één database komt naar voren dat een klant een hypotheek heeft, de andere database zegt van niet. Hoe verwerken we deze tegenstrijdige informatie in het datawarehouse?

Het tweede probleem ontstaat doordat andere processen met een andere bril naar hetzelfde kijken. Bij de afdeling inkoop worden producten gecategoriseerd naar leverancier. Bij de afdeling transport naar gewicht en volume. In het datawarehouse blijkt het soms moeilijk of onmogelijk om op beide manieren naar producten te kijken. Vaak wordt voor de ETL één database gekozen als leading waardoor misschien informatie uit andere systemen verloren gaat. Erger is wellicht dat eindgebruikers het datawa-

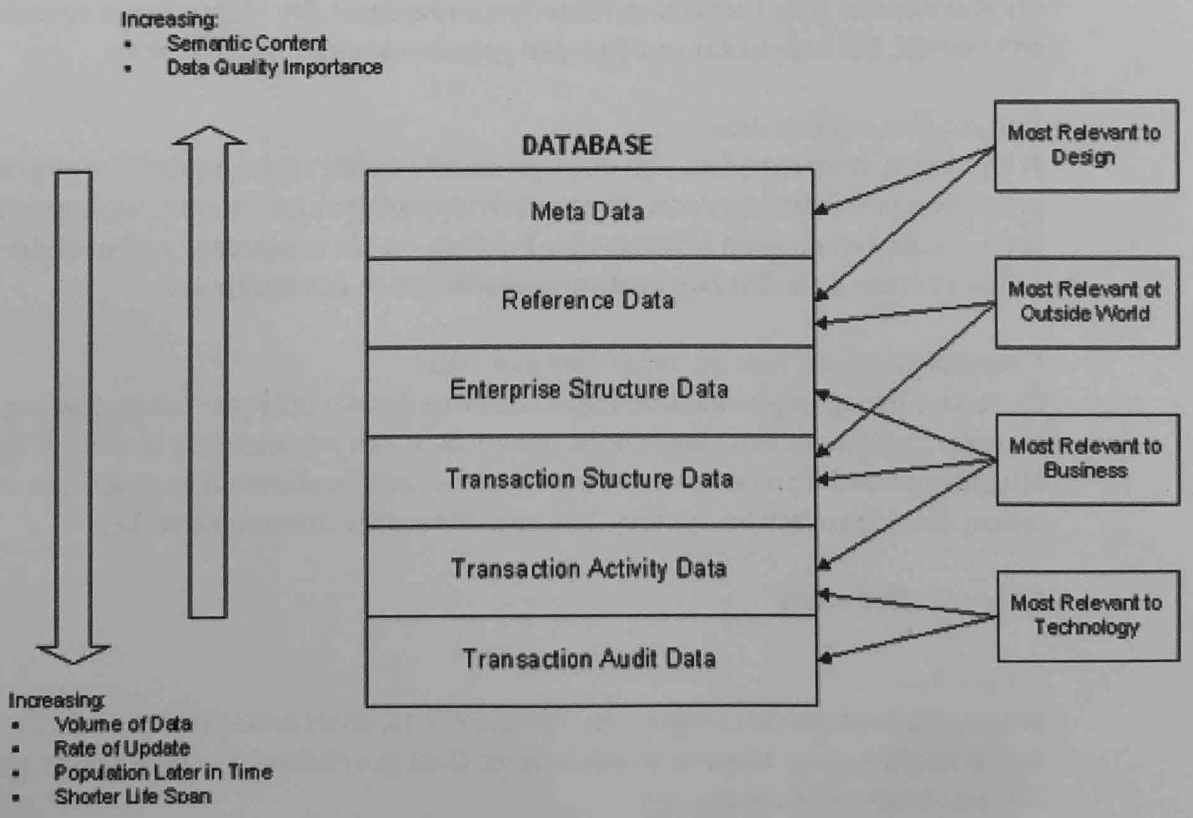
rehouse niet kunnen/willen gebruiken door de in hun ogen verkeerde weergave van zaken.

Master Data Management is een poging dergelijke problemen aan te pakken. Voordat we Master Data en Master Data Management zullen definiëren, loont het om eens nader te kijken naar het soort en type gegevens dat we hebben en moeten beheren.

6.2.1 Verschillende soorten gegevens

Figuur 6.1 (Malcolm Chisholm, *b-eye.network.com*) zet de verschillende soorten gegevens die in een database zitten op een rij. Verschillende soorten gegevens spelen een andere rol en moeten daarmee anders beheerd worden. Van boven naar beneden zie je:

- Metadata
- Reference data
- Enterprise structure data
- Transaction structure data
- Transaction activity data
- Transaction audit data



Figuur 6.1 *Soorten gegevens*

Metadata

Metadata zijn gegevens over de gegevens. Met andere woorden, de metagegevens beschrijven de overige gegevens. Welke kolommen zijn er, wat is hun naam, in welke tabel zitten ze, wat is hun betekenis en welke mogelijke waarden (domein) kunnen

Dit alles leidt ertoe dat er meer klantenlijsten of klantentabellen bestaan binnen één organisatie. Dit heeft twee nadelige gevolgen:

- Gegevens komen niet overeen.
- Definities zijn niet eenduidig.

Een verhuizing is in de ene database wel verwerkt en in een andere niet. Voor dezelfde klant bestaan nu twee adressen waardoor een deel van de post naar het verkeerde adres gaat en de klant nooit bereikt.

De definitie van het begrip klant is niet eenduidig. In de marketingdatabase staan vooral kolommen die iets zeggen over demografie. Wat is de geboortedatum van de klant, welk opleidingsniveau heeft hij of zij, heeft de klant kinderen, et cetera. Op de afdeling financiën is een klant gedefinieerd met kolommen als bankrekeningnummer en creditcardnummer.

Los van de verschillende gegevens die van klanten bijgehouden worden, worden ze ook anders ingedeeld. Vaak zijn er hiërarchieën die op rapporten als drill-downpaden gebruikt worden om van grove bij gedetailleerde informatie te komen. Elke tabel zal zijn eigen hiërarchieën kennen.

Master Data is eigenlijk het idee dat er voor een organisatie maar één lijst van klanten is. Logisch gezien doet een organisatie zaken met een lijst van mensen. Dat zijn dé klanten. Als we Master Data betrekken op klanten, wordt het ook wel Customer Data Integration genoemd. Master Data is echter veel breder. Ook gegevens over producten, grootboekrekeningen, medewerkers en leveranciers zijn voorbeelden van Master Data. Wat hier wordt omschreven als Master Data komt sterk overeen met dimensies in een datawarehouse.

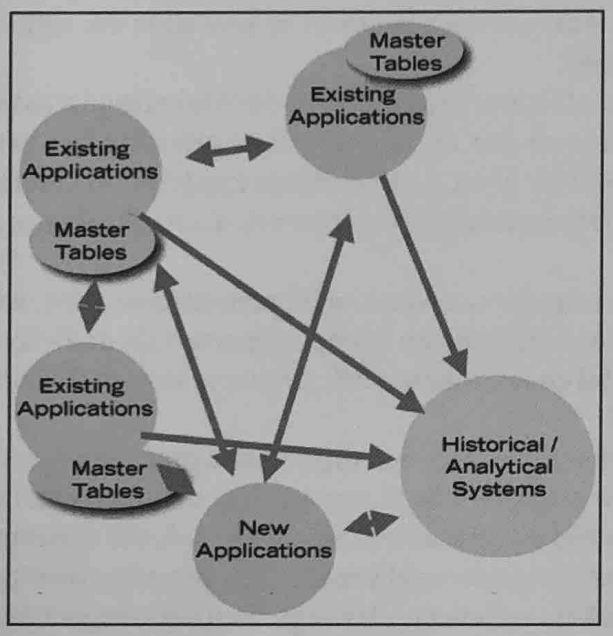
Master Data is de combinatie van reference data, enterprise structure data en transaction structure data. Een voorbeeld van Master Data zijn klantgegevens waar in het ideale geval in een organisatie één centrale set wordt bijgehouden die bepalend is voor alle processen.

Master Data Management

Op het moment dat je het belang van Master Data inziet en initiatieven gaat ontwikkelen om centrale opslag van deze gegevens te realiseren, ben je bezig met Master Data Management (MDM). De definitie van MDM is: '*Master Data Management (MDM) is a set of disciplines and processes for ensuring the accuracy, completeness, timeliness and consistency of the most important types (or domains) of enterprise data across different applications, systems and databases, multiple business processes, functional areas, organizations, geographies and channels.*' (Dan Power, information-management.com).

Master Data Management is een verzameling disciplines en processen die zorgt voor accurate, complete, tijdige en consistente gegevens voor de belangrijke entiteiten binnen een organisatie over verschillende databases, afdelingen en landen heen.

Het tweede deel van deze definitie is eigenlijk een herhaling van wat Master Data is. Het eerst deel (*'a set of disciplines and processes'*) geeft aan dat Master Data Management, net als Business Intelligence, geen eenmalig project is. Het is een continu proces om een centrale en correcte kopie te krijgen en te houden van de Master Data. Bovendien blijkt uit de definitie dat MDM niet los gezien kan worden van datakwaliteit.



6.2.3 Implementatie

Figuur 6.2 *Master Data many-to-many*

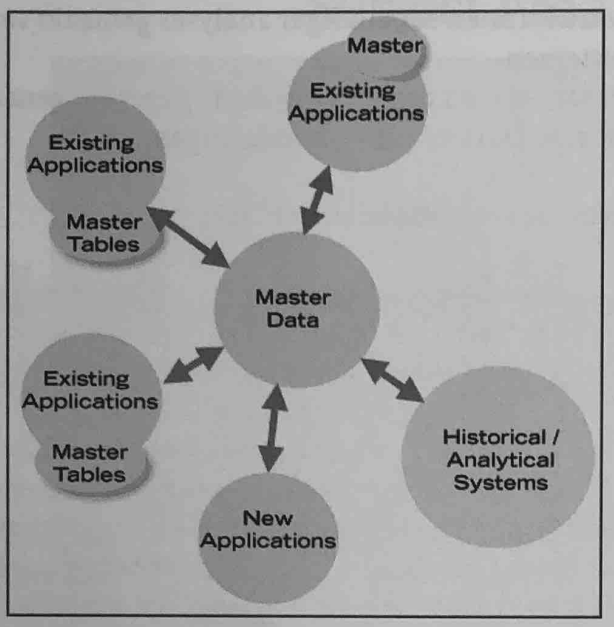
Figuur 6.2 schetst het probleem dat Master Data Management moet oplossen. Een organisatie heeft meer applicaties die elk hun eigen set van Master Data bevatten. In het plaatje zijn dat drie verschillende systemen. Bij het vullen van het datawarehouse moet gekozen worden welk systeem van deze drie leidend is. Als je die keuze niet wilt maken, moet je tijdens het laden eventuele verschillen gladstrijken. In de praktijk is dat een onmogelijke taak.

Figuur 6.3 laat zien waar je heen wilt. Centraal staat de Master Data. De centrale Master Data is leidend bij het vullen van het datawarehouse. Zelf wordt de Master Data gevoed vanuit de applicaties. De applicaties worden echter weer gesynchroniseerd met de centrale Master Data. De centrale Master Data is leidend als er tijdens die synchronisatie verschillen optreden.

Vanwege de centrale rol van de Master Data wordt er vaak gesproken over een Master Data hub.

Let wel op dat MDM niet een project is en al helemaal geen ICT-project. Het is een multidisciplinair proces waarin data governance centraal staat. MDM is eigenlijk een onderdeel van het veel grotere data governance, waarbij 'governance' vertaald zou kunnen worden als 'beleid'.

Data governance staat voor het beleid dat een organisatie voert ten aanzien van gegevens en informatie.



Figuur 6.3 *Master Data one-to-many*

Als onderdeel van data governance moet een organisatie nadenken over hoe om te gaan met gegevens. Wetgevers en toezichthouders stellen daar regels voor op waar je je aan moet houden. Dat wordt compliancy genoemd. Een belangrijk deel van data governance is te zorgen dat je compliant bent. Zaken als beveiliging, verantwoordelijkheid en auditing horen daar ook bij. Master Data Management kan hier een belangrijke rol in spelen.

Compliancy is het voldoen aan (wettelijke) regels en eisen opgelegd door externe partijen zoals overheden en toezichthouders.

Data governance en MDM zijn relatief nieuw en veel projecten om MDM te implementeren mislukken (nog). Voor een deel is de reden hetzelfde als bij Business Intelligence: er wordt te veel naar gekeken als een ICT-project. De technische implementatie is maar een klein deel van een MDM-implementatie. Omdat het multidisciplinair is en deels gaat om eigenaarschap en verantwoordelijkheid, is er een groot politiek aspect aan een MDM-implementatie.
Belangrijke zaken voor een goede introductie van MDM in een organisatie zijn:

- een projectleider die politieke gevoeligheden ziet en begrijpt en die oplossingen daarvoor kan aandragen;
- sponsorship van het management;
- een goede Business Case met een heldere doelstelling die gecontroleerd kan worden;
- duidelijke scope van het (initiële) project;
- inzicht in het businessmodel.

Business Case en doelstelling

Master Data Management kan op drie punten voordelen opleveren:

- Op operationeel niveau worden er minder fouten gemaakt en/of kan er efficiënter gewerkt worden, als de gegevens consistenter zijn.
- Op Business Intelligence-niveau kunnen eenduidiger analyses gemaakt worden van gegevens uit verschillende systemen.
- Het is makkelijker te voldoen aan wet- en regelgeving als de gegevens centraal staan met duidelijke regels omtrent eigenaarschap en veranderingen.

Concreet zou dit kunnen leiden tot de volgende business case:

Voorbeeld

Een bedrijf dat hardware van verschillende merken verhuurt en verkoopt, levert computers aan klanten op basis van specificaties van de klant. Regelmatig treden daar problemen op omdat van processoren verschillende specificaties de ronde doen. Van een Intel Core i7 processor kun je de specificaties vinden bij Intel, maar ook bij leveranciers van notebooks en ook bij veel vergelijkingssites. Wat een processor levert kan ook nog eens verschillen per moederboard. Bij problemen moeten geleverde spullen omgeruild worden, wat tijd en geld kost. Door met één zet van centrale specificaties van hardware te gaan werken, kan het aantal omgeruilde leveringen teruggedrongen worden. De doelstelling is een reductie van 25% in het eerste jaar.

Scope

Net zoals bij een Business Intelligence-project is het van belang de scope van een MDM-implementatie goed te definiëren. Big Bang-implementaties waarbij alle Master Data tegelijkertijd wordt aangepakt zullen mislukken. De hoeveelheid aan gegevens, maar vooral de hoeveelheid aan betrokken mensen en processen, is te groot. Kleine implementaties zijn overzichtelijk en hebben daarmee een kans van slagen. Bovendien kan het nut van MDM daarmee worden bewezen, wat vanzelf leidt tot verdere implementaties.

De scope kan bijvoorbeeld zijn alleen een Master Data-set te maken van productgegevens.

Figuur 6.4 zou kunnen helpen bij het bepalen van de scope. De afbeelding komt overeen met plaatjes die je eerder in het boek hebt gezien. In een matrix zijn complexiteit uitgezet tegen meerwaarde waarbij de grootte van de bubble ook nog de hoeveelheid gegevens aangeeft. Het is verstandig te beginnen bij de kleine bubbles en bij de gebieden die ingeschat worden als gemakkelijk en met veel toegevoegde waarde.

Gedeeltelijke MDM-implementaties kunnen ook overwogen worden. Stel dat afdeling A vijftien kolommen bijhoudt per product, terwijl afdeling B er dertig heeft. Tien van deze kolommen blijken hetzelfde te zijn. Er bestaat echter grote weerstand op de afdelingen om 'hun' informatie over te dragen aan een centraal MDM/data governance-team. Er zou (in eerste instantie) gekozen kunnen worden de overeenkomstige

Business Case en doelstelling

Master Data Management kan op drie punten voordelen opleveren:

- Op operationeel niveau worden er minder fouten gemaakt en/of kan er efficiënter gewerkt worden, als de gegevens consistenter zijn.
- Op Business Intelligence-niveau kunnen eenduidiger analyses gemaakt worden van gegevens uit verschillende systemen.
- Het is makkelijker te voldoen aan wet- en regelgeving als de gegevens centraal staan met duidelijke regels omtrent eigenaarschap en veranderingen.

Concreet zou dit kunnen leiden tot de volgende business case:

Voorbeeld

Een bedrijf dat hardware van verschillende merken verhuurt en verkoopt, levert computers aan klanten op basis van specificaties van de klant. Regelmatig treden daar problemen op omdat van processoren verschillende specificaties de ronde doen. Van een Intel Core i7 processor kun je de specificaties vinden bij Intel, maar ook bij leveranciers van notebooks en ook bij veel vergelijkingssites. Wat een processor levert kan ook nog eens verschillen per moederboard. Bij problemen moeten geleverde spullen omgeruild worden, wat tijd en geld kost. Door met één zet van centrale specificaties van hardware te gaan werken, kan het aantal omgeruilde leveringen teruggedrongen worden. De doelstelling is een reductie van 25% in het eerste jaar.

Scope

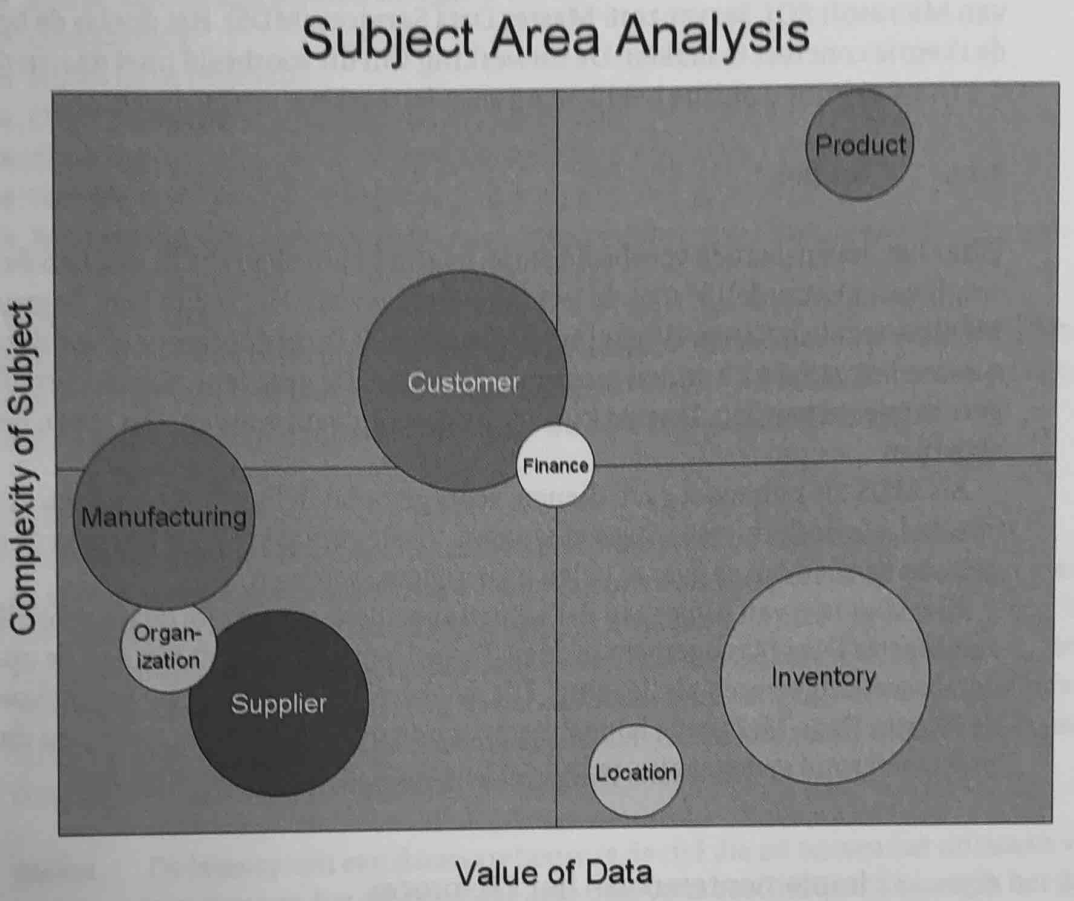
Net zoals bij een Business Intelligence-project is het van belang de scope van een MDM-implementatie goed te definiëren. Big Bang-implementaties waarbij alle Master Data tegelijkertijd wordt aangepakt zullen mislukken. De hoeveelheid aan gegevens, maar vooral de hoeveelheid aan betrokken mensen en processen, is te groot. Kleine implementaties zijn overzichtelijk en hebben daarmee een kans van slagen. Bovendien kan het nut van MDM daarmee worden bewezen, wat vanzelf leidt tot verdere implementaties.

De scope kan bijvoorbeeld zijn alleen een Master Data-set te maken van productgegevens.

Figuur 6.4 zou kunnen helpen bij het bepalen van de scope. De afbeelding komt overeen met plaatjes die je eerder in het boek hebt gezien. In een matrix zijn complexiteit uitgezet tegen meerwaarde waarbij de grootte van de bubble ook nog de hoeveelheid gegevens aangeeft. Het is verstandig te beginnen bij de kleine bubbles en bij de gebieden die ingeschat worden als gemakkelijk en met veel toegevoegde waarde.

Gedeeltelijke MDM-implementaties kunnen ook overwogen worden. Stel dat afdeling A vijftien kolommen bijhoudt per product, terwijl afdeling B er dertig heeft. Tien van deze kolommen blijken hetzelfde te zijn. Er bestaat echter grote weerstand op de afdelingen om 'hun' informatie over te dragen aan een centraal MDM/data governance-team. Er zou (in eerste instantie) gekozen kunnen worden de overeenkomstige

kolommen op te nemen in een MDM-oplossing en de overige kolommen (voorlopig) te laten waar ze zijn.



Figuur 6.4 *Mogelijke aandachtsgebieden bij MDM*

Businessmodel

Van de gegevens die opgenomen gaan worden in een MDM-oplossing moet uitgezocht worden waar ze vandaan komen en welke regels ervoor gelden. Dat kunnen extern opgelegde eisen zijn, maar ook interne businessregels. Achterhaald moet worden wie deze regels kent en wie verantwoordelijk is, danwel gaat zijn, voor de naleving van deze regels.

Met bovenstaand voorbeeld zeg je eigenlijk dat er een data governance-proces opgezet moet worden. Misschien houdt dat wel in dat er een team samengesteld moet worden. Mensen in zo'n team worden vaak data stewards genoemd. Dat zijn mensen met inhoudelijke kennis van zaken die verantwoordelijk gemaakt worden voor de gegevens.

Data stewards zijn mensen met inhoudelijke kennis van zaken die verantwoordelijk zijn voor de gegevens.

6.2.4 Voorbeeld met Microsoft MDS

In deze paragraaf zullen we een eenvoudig voorbeeld uitwerken van MDM op basis van Microsoft SQL Server 2016 Master Data Services (MDS). Het doel is de bovenstaande theorie concreet te maken. De uitwerking van dit voorbeeld (met daarin de figuren 6.5 t/m 6.7) kunt u online bekijken op *www.leerboekbusinessintelligence.nl.*

6.2.5 Tot slot

Waar het bovenstaande voorbeeld stopt, begint het in de praktijk pas. Een data steward wordt verantwoordelijk voor de leveranciersgegevens. Het is zijn taak de gegevens in MDS correct te houden. Wijzigingen kunnen met Excel doorgevoerd worden. Ook nieuwe leveranciers kunnen toegevoegd worden. De gegevens moeten vervolgens gecontroleerd worden. Daarna kunnen ze geaccepteerd worden. Dat is een belangrijke stap hier.

Als MDS als hub moet gaan dienen, zoals getoond in figuur 6.3, moeten er synchronisatiemechanismes opgezet worden. Via de gemaakte view kunnen de gegevens gelezen worden om ze over te halen naar andere systemen.

Hiervoor is ervan uitgegaan dat we een specifieke set van software gebruiken om aan Master Data Management te doen. In veel bedrijven wordt één van de operationele databases aangewezen als 'leading'. Die database bevat dan wat we beschouwen als de Master Data. Met goede koppelingen tussen de verschillende systemen en goede processen rond synchronisatie kan dat uiteraard ook een oplossing zijn.

6.3 Implementeren van het ETL-proces

Na het uitstapje naar Master Data Management gaan we nu verder kijken naar het ETL-proces zelf. Ook voor het ETL-proces geldt, net als voor alle andere onderdelen van Business Intelligence, dat de informatieanalyse leidend moet zijn bij het nemen van ontwerpbeslissingen. Er moeten beslissingen genomen worden over welke gegevens overgehaald moeten worden, maar ook over de architectuur en de frequentie van het ETL-proces. Daarnaast moeten beslissingen genomen worden over de technische implementatie. Ga je de taal SQL (Structured Query Language) gebruiken om gegevensmanipulatie (de T van ETL) te doen, of gebruik je een van de vele beschikbare tools?

6.3.1 Resultaten van informatieanalyse

Uit de informatieanalyse moet een aantal inhoudelijke zaken duidelijk worden. Daarnaast moet ook een aantal meer technische zaken helder worden. De inhoudelijke zaken hebben betrekking op de gegevens zelf, maar ook over hoe up-to-date rapportages moeten zijn, hoe met wijzigingen van gegevens wordt omgegaan – zowel bij dimensiegegevens als ook bij feiten – en hoe lang gegevens in het datawarehouse (of datamart) beschikbaar moeten blijven. De technische aspecten hebben betrekking op het auditen en monitoren van de ETL en het omgaan met foutsituaties.

Onderstaand worden de volgende aspecten van ETL die in de informatieanalyse aandacht dienen te krijgen, kort belicht:

- Welke kolommen moeten meegenomen worden?
- Data latency
- Historische gegevens
- Oude gegevens
- Auditing
- Fouten
- Monitoring

Welke kolommen

Duidelijk is dat de informatiebehoeften (welke informatie moet op welke rapporten beschikbaar worden) stuurt welke kolommen overgehaald moeten worden naar het datawarehouse. In paragraaf 6.3.3 zullen we daar iets meer over zeggen.

Data latency

Het tweede punt waar aandacht aan besteed moet worden tijdens de informatieanalyse is vragen met betrekking tot het realtime zijn van gegevens. Wanneer zien gebruikers wijzigingen die zij aanbrengen in hun operationele applicaties terug in de rapporten? Als iemand op personeelszaken een nieuwe ziektemelding invoert, zien we die ziekte-uren dan meteen terug op het dashboard dat aangeeft of we qua ziekteverzuim op schema liggen om de doelstelling te halen? Of komt deze nieuwe ziektemelding pas morgen of overmorgen op de rapporten terecht?

De **latency** van een datawarehouse is de tijd die zit tussen het ontstaan van gegevens en het moment dat deze gegevens beschikbaar komen in het datawarehouse.

De eisen die gesteld worden aan de latency hebben in eerste instantie betrekking op de frequentie waarmee het ETL-proces uitgevoerd moet worden. Bij veel Business Intelligence-oplossingen zien we dat de ETL elke nacht wordt uitgevoerd. De latency is daarmee een dag. Gegevens die vandaag ontstaan, zijn pas morgen terug te vinden in het datawarehouse.

Voor de meeste managementinformatie (stuurinformatie) is het niet erg als de gegevens een dag oud zijn. Steeds vaker willen mensen echter (near) realtime-informatie en is een latency van een dag niet meer aanvaardbaar. Alles is mogelijk. Wel worden de oplossingen die nodig zijn complexer en duurder.

Historische gegevens

In paragraaf 3.7 is al het ontbreken van historische gegevens in veel operationele databases besproken. In paragraaf 4.1.4 is besproken hoe Slowly Changing Dimensions (SCD) een technische oplossing bieden om met historische gegevens om te gaan. In het ETL-proces moet je de gekozen manier om met veranderende gegevens om te gaan implementeren. Voor elke kolom die vanuit het bronsysteem wordt overgehaald, moet bekend zijn wat gedaan moet worden met gewijzigde waarden. Dit moet uit de infor-

matieanalyse duidelijk worden. Vervolgens moet bedacht worden hoe je erachter komt of waarden gewijzigd zijn. Tot slot moet het juiste type SCD geïmplementeerd worden.

Naast dimensies die kunnen veranderen, kunnen ook records in feitentabellen veranderen. In paragraaf 4.1.7 is de accumulating snapshot feitentabel besproken. Voor 'gewone' feitentabellen geldt dat je alleen nieuwe records hoeft toe te voegen aan de feitentabel. Bij een accumulating snapshot tabel moeten echter ook al bestaande records gewijzigd worden. Uit de informatieanalyse moet duidelijk worden of dit scenario nodig is.

Naast accumulating snapshots zijn er ook situaties denkbaar waar feiten later aangepast moeten worden. Als je een feit overschrijft met de nieuwe waarde, veranderen rapporten met terugwerkende kracht. Is dat wenselijk?

De organisatie kan eisen dat elk rapport een peildatum krijgt. De waarden die getoond worden zijn de waarden die actueel waren op het moment van de peildatum. Bij een wijziging van het feit moet je een nieuw record aanmaken. Op basis van de peildatum moet dan het juiste record gelezen worden.

Als je van een feit twee records bijhoudt, gaan totaaltellingen niet meer goed. Een record opnemen met een tegenboeking kan dit oplossen.

De moraal van het verhaal is dat feiten achteraf kunnen wijzigen. Uit de informatieanalyse moet blijken hoe de ETL hiermee moet omgaan.

Oude gegevens

Naast historische gegevens in de zin van veranderende gegevens, is het interessant om te weten hoe ver terug in de geschiedenis het datawarehouse of de datamart moet gaan. Is het nog interessant om omzetgegevens van vijftien jaar geleden te bekijken? En zo ja, moeten die dan geïndexeerd worden om te kunnen vergelijken met de gegevens van het huidige jaar?

In paragraaf 5.4 zijn allemaal performancefeatures besproken om query's op datawarehouses sneller te maken. De belangrijkste is misschien wel tabellen klein te houden. Voor een deel gaat dat in tegen het doel van een datawarehouse.

Voor een datamart is het belangrijk te bepalen hoe ver terug in de tijd gegevens vastgehouden moeten worden. Als uit de informatieanalyse blijkt dat alle rapporten en analyses betrekking hebben op het huidige jaar en er hooguit wordt vergeleken met dezelfde periode in het voorgaande jaar, zorg dan dat er niet meer dan twee jaargangen aan gegevens in de datamart aanwezig zijn. Dit houdt in dat er in het totale ETL-proces ook een onderdeel moet zitten dat oude gegevens verwijderd.

Auditing

Alle bovenstaande eisen zijn functioneel van aard. Ze hebben rechtstreeks te maken met hoe mensen welke gegevens gaan zien op de rapportages. Er moet echter ook over een aantal meer technische specificaties worden nagedacht alvorens de ETL te implementeren. Eén daarvan is auditing.

Auditing is het toevoegen van metadata aan de gegevens. In de context van ETL-auditing komt dit neer op het toevoegen van informatie over de ETL zelf aan de gegevens die geladen worden. Meestal gaat het dan over het laden van feiten, maar auditing hoeft zeker niet beperkt te blijven tot feiten. Van elk geladen feit wordt bijgehouden

wanneer het is geladen, door wie, door welk proces (package) en welke versie, en welke transformaties zijn uitgevoerd op de gegevens voor het laden.

Auditing is het toevoegen van metadata aan de gegevens in het datawarehouse zodat is te herleiden wie of welk proces de gegevens wanneer en hoe heeft geladen.

Door audit-informatie toe te voegen als dimensie aan het datawarehouse, kunnen vragen beantwoord worden als:
- Waar komt deze informatie vandaan (data lineage)?
- Hoe komen we aan deze getallen?

Audit-informatie hoeft niet per se als dimensie te worden toegevoegd. Andere manieren zijn ook denkbaar.

Auditing is cruciaal als je moet voldoen aan compliance-eisen. In steeds meer situaties moet je voldoen aan wetten er regels. Accountants eisen dat je precies kunt herleiden hoe en waar informatie tot stand is gekomen. Auditing is daarmee vaak een must. Uit de informatieanalyse moet naar voren komen aan welke compliance-eisen voldaan moet worden.

Fouten

ETL is software en bij software moet je nadenken over hoe je wilt omgaan met foutsituaties. Het kan bijvoorbeeld voorkomen dat je een numeriek gegeven als tekst krijgt aangeleverd. Tijdens de conversie van tekst naar numeriek blijken er toch alfanumerieke karakters voor te komen bij enkele records. Hoe moet het ETL-proces hiermee omgaan? Vervangen we de niet te vertalen gegevens naar 0 of wellicht naar NULL en wordt het record gewoon geladen? Of wordt het hele record geweigerd en bijvoorbeeld voor handmatige correctie in een aparte tabel weggeschreven?

Monitoring

Beheerders van ETL-processen moeten weten of de processen hebben gedraaid, of ze succesvol of met een fout zijn geëindigd, wat de totale doorlooptijd van het proces was en misschien nog wel andere zaken. Zorg daarom dat er in de informatieanalyse ook aandacht voor dit soort zaken is.

6.3.2 Architectuur

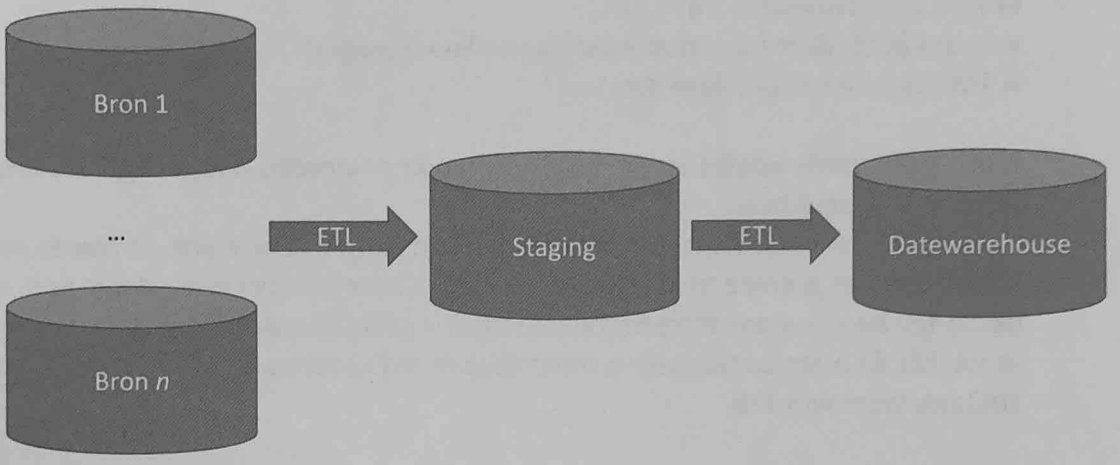
Uit de informatieanalyse komen allemaal eisen naar voren die van invloed zijn op de opzet van de ETL, zoals in de vorige paragraaf duidelijk is geworden. Naast deze eisen is er nog een aantal factoren die een rol spelen:
- de hoeveelheid gegevens (in MB/GB/TB) die per keer overgehaald moeten worden;
- het aantal en soort bronnen;
- tijdstip en tijdsduur dat gegevens uit de bronnen ingelezen mogen worden.

Deze factoren bepalen welke architectuur het beste is. Grofweg kunnen er drie architecturen worden onderkend, maar er zijn vele variaties mogelijk. In hoofdlijnen zijn de oplossingen:

- zonder staging-laag;
- met staging;
- met twee staging-lagen.

De keuze is ook nog afhankelijk van de methodiek: Kimball, Inmon of Data Vault.

Geen staging



Figuur 6.8 *Architectuur zonder staging database*

Figuur 6.8 laat de architectuur zien zonder staging database. Een staging database of staging-laag is een plek waar de gegevens tijdelijk worden geparkeerd alvorens ze worden overgebracht naar het datawarehouse.

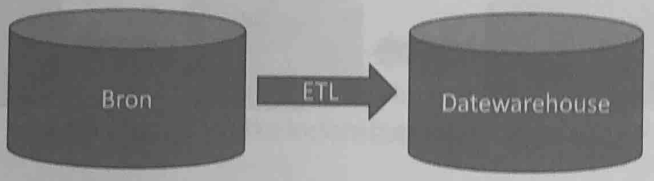
Een **staging database** is een database waar gegevens tijdelijk worden opgeslagen alvorens ze worden doorgestuurd naar het datawarehouse.

De architectuur zonder staging wordt niet veel gebruikt. Die is vooral geschikt als je weinig (misschien maar één) bronsystemen hebt en de hoeveelheid gegevens die overgehaald moeten worden beperkt is. Je hebt de hele nacht de tijd om de ETL te doen, gedurende die hele tijd kun je doen en laten wat je wilt met zowel het bronsysteem en het datawarehouse. Als er iets fout gaat tijdens de nacht kun je de ETL overdag tijdens kantooruren opnieuw draaien. Ook met een inefficiënte aanpak is de ETL gemakkelijk klaar voordat de eerste mensen op kantoor komen en rapporten gaan gebruiken. Dit scenario is niet ondenkbaar, maar het zijn wel een heleboel als-en bij elkaar.

Eén staging database

Figuur 6.9 laat een architectuur zien met één staging database. Deze architectuur geeft veel meer flexibiliteit en is daarmee ook een veel toegepaste architectuur. In een eerste stap worden de benodigde gegevens uit de verschillende bronnen gehaald en in de staging database gezet. De structuur van de staging database lijkt voornamelijk op die

van de bronsystemen. Het belangrijkste is om de gegevens zo efficiënt mogelijk uit de bronsystemen te halen.



Figuur 6.9 *Architectuur met staging database*

Vaak zijn er eisen met betrekking tot de tijden waarop de ETL de bronsystemen mag benaderen. De ETL heeft een nadelige impact op de performance van andere workloads die tegelijkertijd plaatsvinden op de bron. Je kunt dan denken aan bijvoorbeeld het maken van back-ups en het reorganiseren van indexen op de bronsystemen. Deze zaken vinden vaak net als de ETL 's nachts plaats. Deze werkzaamheden zorgen er (soms) voor dat je beperkt toegang krijgt tot de brondatabases (*limited window of opportunity*).

Met meer bronnen zou het ook nog eens zo kunnen zijn dat je één bron al vroeg in de avond moet benaderen, terwijl een andere bron pas later in de nacht beschikbaar is. In een 24/7-omgeving komt daar nog bij dat de ETL een zo min mogelijke impact moet hebben op het primaire proces. Een staging database helpt bij al deze overwegingen.

Tijdens het laden van het datawarehouse kan er altijd iets fout gaan. De vraag is dan: wat doe je als je er 's ochtends achter komt dat het datawarehouse niet (correct) geladen is? Moet je de gebruikers vertellen dat ze een hele dag met rapporten moeten werken gebaseerd op oude gegevens, of kun je de ETL opnieuw starten zodat in de loop van de ochtend alsnog up-to-date rapporten beschikbaar komen? Met de gegevens in staging is er een grote kans dat je de ETL kunt herstarten.

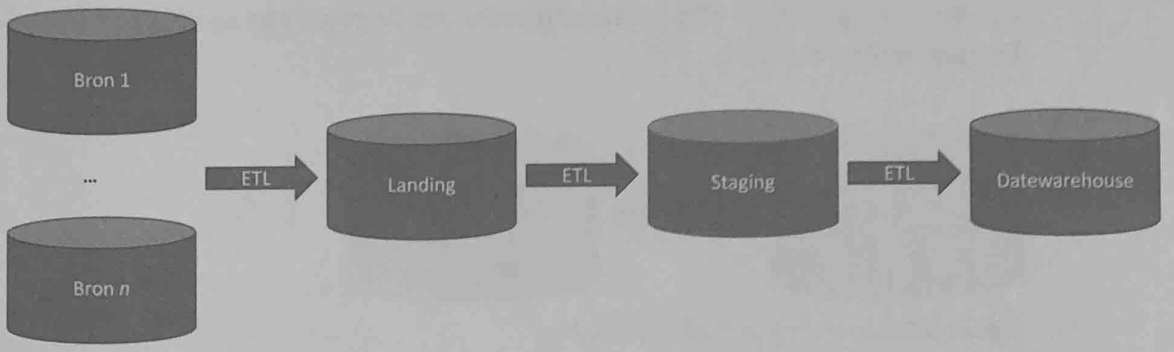
De voordelen van het gebruik van een staging database zijn:

- meer flexibiliteit;
- herstarten van de ETL is gemakkelijker;
- omgaan met verschillende databases en verschillende beschikbaarheid van databases wordt gemakkelijker;
- in staging kunnen bewerkingen uitgevoerd worden zoals uitzoeken welke gegevens gewijzigd zijn.

De ETL wordt nauwelijks complexer door het gebruik van een staging database. Daarom zien we dat deze architectuur veel gekozen wordt.

Twee staging databases

In figuur 6.10 zie je dat er nog een extra database is toegevoegd aan de architectuur, de landing database. De staging database in de architectuur van figuur 6.9 lijkt qua opzet voornamelijk op de bronsystemen. Dat houdt in dat de meeste transformaties om de gegevens te krijgen in de structuur van het datawarehouse plaatsvinden in de stap van staging naar datawarehouse. Dit kan invloed hebben op de laadsnelheid van het datawarehouse.



Figuur 6.10 *Architectuur met twee staging databases*

Bij een multinational met kantoren over de hele wereld is er altijd wel ergens een kantoor open. Vanaf dat kantoor worden corporate rapporten opgevraagd die gebaseerd zijn op het datawarehouse. Het datawarehouse is dus een 24/7-database geworden. Dat houdt in dat de impact van het verversen geminimaliseerd moet worden.

Door een extra laag in te bouwen, kun je een groot deel van de tijdrovende transformaties doen tussen landing en staging database. De landing database lijkt nu qua ontwerp op de bronnen, de staging lijkt juist op het datawarehouse. Het vullen van het echte datawarehouse is nu het een-op-een overhalen van gegevens uit staging naar datawarehouse.

De voorgaande discussie over wel of geen staging gaat min of meer uit van een op de Kimball-methode gebaseerd datawarehouse. Voor Inmon en Data Vault gelden echter grotendeels dezelfde argumenten. Een belangrijk verschil is wel dat een Inmon-datawarehouse of een Data Vault nooit rechtstreeks gebruikt worden. Er is nog een ETL-slag nodig van het datawarehouse naar de verschillende datamarts. De datamarts zijn de databases die gebruikt worden voor rapportages en analyses.

De landing database hoeft niet perse een database te zijn. Je kan er ook voor kiezen vanuit bronsystemen exports te doen naar flat files. Deze flat files vormen dan als het ware de landingzone. Deze files kunnen geïmporteerd worden naar staging. Vanaf staging gaan ze door naar het datawarerhouse. Deze opzet ontkoppelt' de BI-oplossing van de bronsystemen waardoor meer flexibiliteit ontstaat bij veranderingen aan de bronnen.

Fysieke architectuur

Bovenstaand verhaal schetst de logische architectuur. Je moet ook nog nadenken over de fysieke architectuur. Is elke laag ook echt een aparte database? En zo ja, staat elke database op een aparte server?

In kleinere omgevingen zie je nog wel eens dat wat hier de staging database wordt genoemd, gewone tabellen zijn in het datawarehouse. In het geval van twee staging databases kan het best zijn dat landing en staging andere tabellen in dezelfde database zijn. Grootte, belasting van de database, en duurzaamheid van de inhoud bepalen wat wijsheid is.

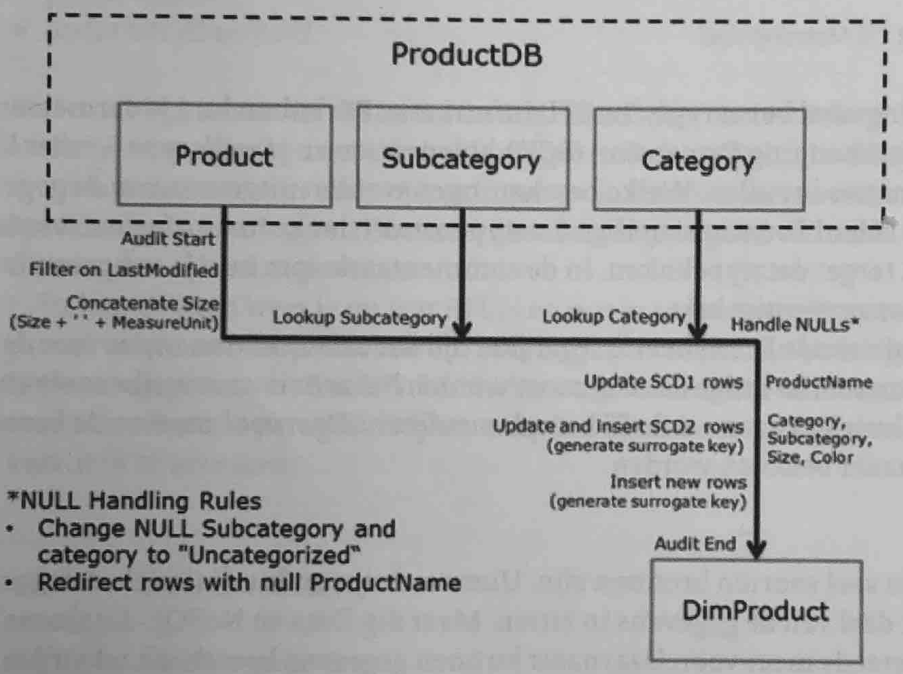
Of verschillende databases op dezelfde server kunnen draaien of niet, hangt af van de overall workload van die server. Vaak zal staging overdag helemaal niets doen waarmee het de performance van het datawarehouse niet in de weg zit. Er is dan niets mis

met het combineren van beide op dezelfde server. Bij een 24/7-datawarehouse wordt dat natuurlijk anders.

6.3.3 Documenteren

Tot nu toe is in algemene termen over de ETL gesproken. We zullen nu iets gedetailleerder kijken. Welke kolommen worden gebruikt?

Kolom-mapping



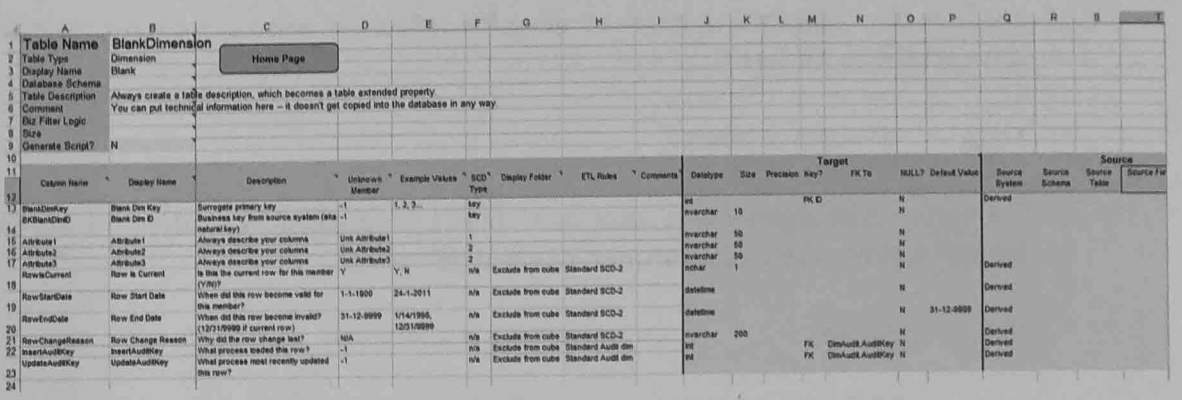
Figuur 6.11 *ETL Mappingdiagram*

Figuur 6.11 is een mappingdiagram die weergeeft hoe de dimensie dimProduct gevuld wordt. Er zijn twee redenen om mappingdiagrammen te maken:

- om helder te krijgen welke brontabellen nodig zijn;
- om de ETL te documenteren.

Niet elke datawarehousekolom is een-op-een hetzelfde als een kolom uit een bronsysteem. Het is daarom van belang zaken met betrekking tot validaties en transformaties op te nemen. Omdat het hier om een high level-diagram gaat, mist de detailinformatie. In een mappingtabel, vaak een Excel-document, kun je in detail aangeven welke kolom uit het bronsysteem op welke manier in het doelsysteem terechtkomt.

Figuur 6.12 is een voorbeeld van een Excel-spreadsheet met gedetailleerde mappinginformatie. Het hier getoonde workbook komt van Kimball en is te downloaden van zijn site. De eerste kolommen beschrijven de kolommen in het doelsysteem. Als dit het datawarehouse is, kunnen deze dienen als documentatie. De sheet van Kimball heeft zelfs een macro die de SQL DDL (Data Definition Language) genereert om de tabel te maken. Daartoe gebruikt de sheet de datatype-informatie die je kunt/moet opgeven. Ook informatie over foreign keys wordt opgenomen.



| | | |

SQL

Los van welke tool je kiest, is het nog wel de moeite waard stil te staan bij de kracht van SQL. Daar waar gegevens uit relationele databases komen, kun je SQL gebruiken om de gegevens te manipuleren. SQL is een heel krachtige en efficiënte taal om gegevensmanipulatie mee te doen.

Er zijn drie argumenten bij het kiezen van de juiste tool, waarbij SQL een van de mogelijke tools is:

- benodigde functionaliteit: zowel pure ETL-functionaliteit als ook monitoring- en logging-functionaliteit;
- performance;
- onderhoudbaarheid.

Een combinatie van een ETL-tool en SQL is vaak het beste. Wees je bewust van de sterkte en zwakke kanten van beide en gebruik de mogelijkheden slim. Belangrijk is om consequent te zijn in de oplossingen en misschien wel vooraf standaarden af te spreken en vast te leggen. Documenteer vervolgens goed wanneer en waarom je van die standaard bent afgeweken.

In hoofdlijnen weet je nu wat ETL is en wat je moet doen en weten om een ETL-proces te kunnen implementeren. Wat nog nauwelijks aan bod is gekomen is de kwaliteit van gegevens en hoe ETL kan helpen die kwaliteit te verbeteren. Daarom gaan we in de volgende paragraaf kijken naar het opschonen van gegevens om de gewenste kwaliteit te bereiken.

6.4 Datakwaliteit

6.4.1 Slechte gegevens

Als een van de redenen waarom rapporteren en analyseren op operationele databases niet goed werkt, is de kwaliteit van de gegevens in bronsystemen genoemd. Er zijn verschillende redenen dat de kwaliteit van de gegevens niet is wat het zou moeten zijn:

- Er zijn meer bronsystemen. Elk bronsysteem heeft zijn eigen versie van de waarheid.
- Situaties veranderen.
- Gegevens worden foutief en onvolledig ingevoerd.

Meer bronnen

Master Data Management kan een oplossing zijn als dimensiedata in meer bronnen voorkomt. Dat klinkt gemakkelijk, maar dan moet je nog wel zorgen dat de kwaliteit van de Master Data goed is. Zonder MDM-oplossing moet je bij het laden van dimensies iets met de kwaliteit van de gegevens doen.

Situaties veranderen

Naast het probleem van meerdere bronnen blijkt dat de gegevens zelf inherent slecht zijn van kwaliteit. Soms zijn gegevens niet meer waar. De contactpersoon bij een klant krijgt bijvoorbeeld een nieuwe baan zonder dat je dat weet, maar in de database staat hij nog altijd geregistreerd als contactpersoon van zijn oude bedrijf. Misschien dat een

klant failliet is gegaan. De klant kan een preferred supplier-contract hebben afgesloten met een concurrent. Er kan van alles gebeuren buiten het zichtveld van een organisatie, dat ervoor zorgt dat de gegevens niet meer kloppen.

Foutieve invoer
Soms worden gegevens door mensen ingevuld en mensen maken fouten. Typefouten zijn niet te voorkomen. Maar ook als er procedures en best practices zijn met betrekking tot gegevensinvoer, zullen medewerkers er zich niet altijd aan houden. Je zou kunnen afspreken dat als er 'bv' in de naam van een klant voorkomt, je dat altijd opschrijft zonder puntjes en met kleine letters. Het is echter een kwestie van tijd voordat er namen voorkomen met B.V. er in, met puntjes en hoofdletters.

Bij webshops is het probleem nog veel groter. Veel mensen laten dummygegevens achter uit privacyoverwegingen. Anderen maken elke keer dat ze terugkeren naar de webshop een nieuw account aan. Je hebt zelf weinig tot geen controle over deze gegevens.

6.4.2 Data Cleansing

Voor correcte analyses zijn goede gegevens nodig. De gegevens zullen moeten worden opgeschoond. Voor het opschonen van gegevens worden de termen data cleansing, data cleaning en data scrubbing gebruikt. Volgens Wikipedia is data cleansing '*the process of detecting and correcting (or removing) corrupt or inaccurate records from a record set, table or database. The term refers to identifying incomplete, incorrect, inaccurate, irrelevant, etc. parts of the data and then replacing, modifying or deleting this dirty data.*'

Data cleansing is het opsporen en verbeteren of verwijderen van inconsistente en foutieve records uit een verzameling, tabel of database.

In paragraaf 3.5 zijn veel van de mogelijke problemen en de gevolgen van slechte gegevens al besproken. We gaan dat hier niet herhalen. In de volgende paragraaf zullen we een concrete implementatie laten zien van hoe je de kwaliteit van gegevens kunt verbeteren. Gezien het bovenstaande is wel duidelijk dat het hier niet om een eenmalig iets gaat. Als je brongegevens eenmalig opschoont, is het een kwestie van tijd voordat de problemen weer terug zijn. Daarom moet het opschonen een regelmatige terugkerende routine worden. Het is misschien wel de belangrijkste stap van het ETL-proces.

Als je gegevens overhaalt naar een MDM-implementatie of naar datamarts, moet de kwaliteit goed zijn.

6.4.3 Voorbeeld met Microsoft DQS

Om een en ander concreet te maken zullen we een eenvoudig voorbeeld uitwerken. U vindt dit voorbeeld weer online op *www.leerboekbusinessintelligence.nl*.

6.5 Scenario's

In deze laatste paragraaf van dit hoofdstuk zullen we weer met voorbeelden gebaseerd op SQL Server 2016 proberen het verhaal concreet te maken. In eerste instantie zullen we SQL Server 2016 Integration Services gebruiken om een dimensietabel in het datawarehouse te vullen met gegevens uit de brondatabase. Twee van de attributen zijn gedefinieerd als Slowly Changing Dimension type 2 attributen.

In deze paragraaf gaan we ervan uit dat je een server hebt met daarop Microsofts voorbeelddatabases AdventureWorks2014 en AdventureWorksDW2014. In ons geval staan die databases op een server genaamd SQL2016. Overal waar je in voorbeelden, plaatjes of code SQL2016 (of Denali) ziet staan, moet je dat vervangen door de naam van jouw server. Omdat deze paragraaf weer op concrete voorbeelden ingaat die productspecifiek zijn, vind je de rest van de paragraaf op *www.leerboekbusinessintelligence.nl*.

6.6 Tot slot

In dit hoofdstuk heb je in de brede zin van het woord geleerd wat ETL is. In zijn meest basale vorm is het niets anders dan het kopiëren van gegevens van een bronsysteem naar een doelsysteem. In een breder perspectief is het ook het aanpassen van de gegevens aan de eisen van het doelsysteem. Herleidbaarheid, auditing en eventueel compliancy komen dan om de hoek kijken. Het aanpassen zelf kan technisch van aard zijn (bijvoorbeeld datatype-conversies) of meer inhoudelijk.

Datakwaliteit is belangrijk. Met gedistribueerde systemen is Master Data en Master Data Management een uitdaging. Wat is de bedrijfsbrede waarheid ten aanzien van onze klanten, producten, leveranciers et cetera? Daarnaast zit er veel 'rotzooi' in bronsystemen. Voor analyses geldt het al oude adagium: *garbage in, garbage out*. Je moet dus zorgen dat de input die de data-analisten krijgen (de gegevens uit het datawarehouse) geen garbage bevat maar kwalitatief goede gegevens.

Om dit alles te bereiken zijn er veel tools op de markt. Er zullen veel keuzes gemaakt moeten worden. Welke tools en welke combinatie van tools moeten gebruikt worden?

Als het je ten slotte is gelukt om een goed datawarehouse (of een goede datamart) te maken, qua design maar zeker ook qua inhoud, kunnen analisten goede analyses gaan doen. In hoofdstuk 8 gaan we kijken naar analytische modellen. Wat kunnen we nu uiteindelijk doen met al die mooie gegevens? Het is echter eerst tijd voor Big Data.